COMPUTER NETWORKS LAB

BCSE303P

REVIEW - 1 PROJECT REPORT

1. Final Project Title

Intelligent Network Routing Simulator

2. Team Details

S.No.	Student Name	Registration Number
1	Divyansh Mani Tripathi	24BAI0264
2	Shaheli Paul	24BCE0562
3	SaiKaushick K.S	24BAI0265

3. Abstract

Modern Computers rely on routing protocols to determine efficient, reliable paths for data transfer between devices. However, understanding how these protocols behave under different network conditions can be difficult through theory alone. This project aims to develop an Intelligent Network Routing Simulator that models and analyzes two widely used routing protocols: Routing Information Protocol (RIP), which uses the Bellman-Ford algorithm, and Open Shortest Path First (OSPF), which uses Dijkstra's algorithm.

The simulator allows users to create customizable network topologies, generate routing tables, and visualize packet transmission across the network. It also simulates real-world scenarios such as link failures, router failures, congestion, and packet loss to study their effect on network performance. Various metrics such as recovery time, throughput, and delay are measured and compared for both protocols.

In addition to protocol simulation, the project also recommends the protocol for the network based on factors such as network size, traffic load, congestion level, and failure frequency, the system recommends the more suitable routing protocol and provides a reason for the recommendation. To improve reliability, the simulator results are validated using packet captures from Wireshark and by creating equivalent network topologies in Cisco Packet Tracer.

The final project provides a unified platform for routing simulation, protocol comparison, network failure analysis, performance evaluation, and visualization. It serves as both a learning tool for understanding routing concepts and a practical platform for analyzing the behaviour of routing protocols under different network conditions.

4. Problem Statement / Objective

4.1 Problem Statement

Routing protocols play an important role in computer networks by determining the path through which data travels from a source to a destination. Although protocols such as RIP and OSPF are taught mainly through theory and algorithms like Bellman-Ford and Dijkstra. This makes it difficult to understand how these protocols behave in practical situations such as network failures, congestion, or packet loss.

There is a need for an interactive platform where users can create network topologies, observe routing behaviour, compare protocol performance, and understand the impact of different network conditions. In addition, selecting the most suitable routing protocol for a network is often challenging because factors such as network size, traffic load, congestion levels, and failure frequency can significantly affect protocol performance. Most educational tools focus only on simulation and comparison, without providing guidance on protocol selection.

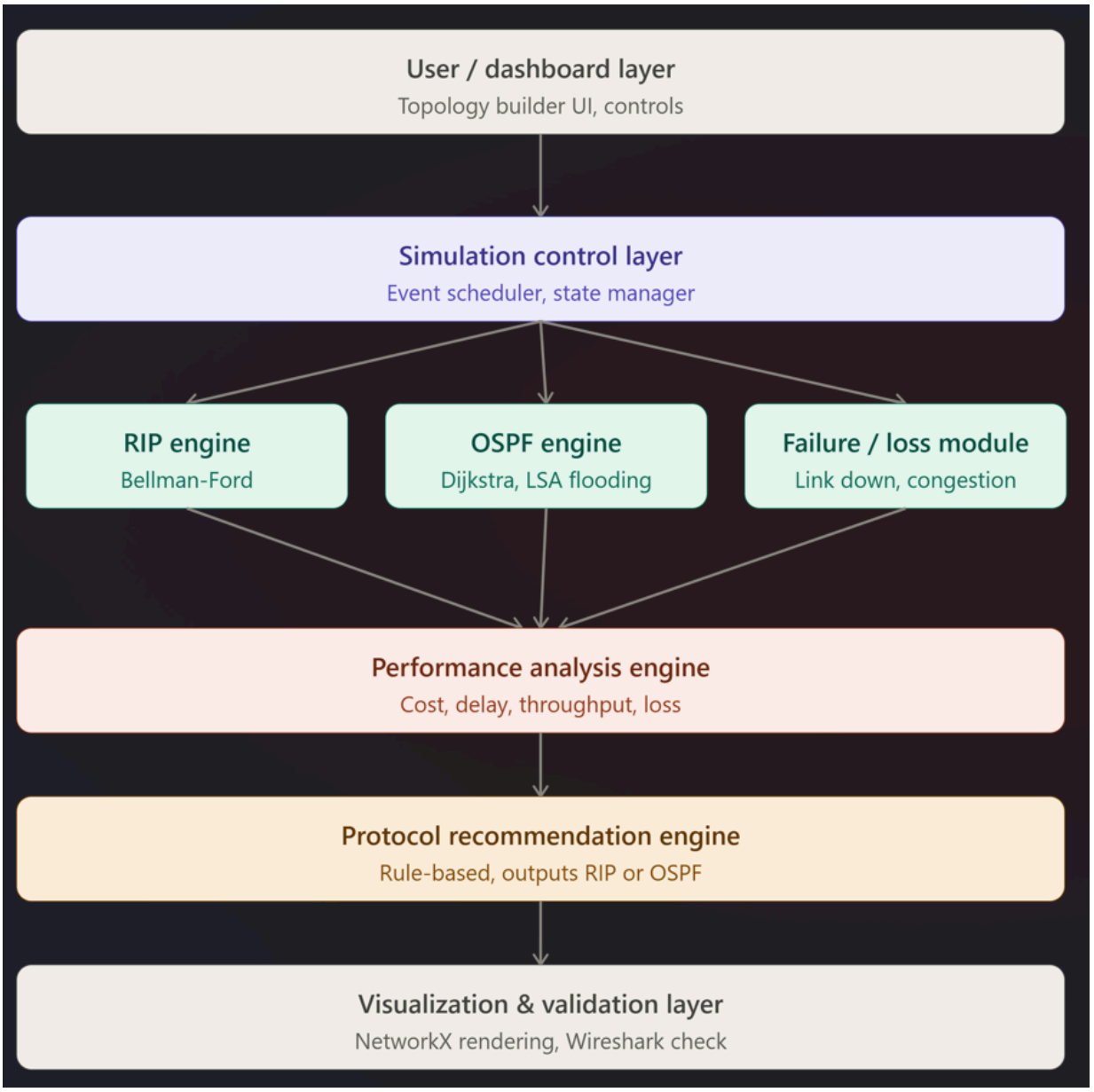
Therefore, a system is required that not only simulates and analyses routing protocols under different network scenarios but also recommends the most appropriate protocol based on network characteristics and performance metrics. This can bridge the gap between theoretical concepts and practical implementation while helping users make informed routing protocol choices.

4.2 Objectives

- Design and simulate network topologies with configurable routers and links.
- Implement RIP using the Bellman-Ford algorithm and OSPF using Dijkstra's algorithm.
- Display network topology, routing tables, and packet flow through a graphical interface.
- Simulate network events such as link failures, router failures, congestion, and packet loss.
- Measure and compare the performance of RIP and OSPF using metrics such as recovery time, throughput, delay, and packet loss.
- Develop a protocol recommendation system that suggests the most suitable routing protocol based on network conditions.
- Validate simulation results using Wireshark packet captures and equivalent network setups in Cisco Packet Tracer.

5. Proposed System Architecture

Our project follows a layered architecture where the network topology and simulation controls form the top layer. The RIP and OSPF protocol modules operate on this topology to generate routing information and determine packet paths. The outputs from both protocols are passed to a common performance analysis module, which evaluates metrics such as convergence time, delay, throughput, recovery time, and packet loss. Based on these results and network conditions, the protocol recommendation engine suggests the most suitable routing protocol for the given scenario. The entire system is supported by visualization components for topology and packet-flow monitoring, while Wireshark and Cisco Packet Tracer are used for external validation of the simulation results. The overall architecture of the proposed system is shown below.



5.1 Layer-wise Explanation

- User / Dashboard Layer: This layer acts as the main interface through which the user interacts with the simulator. It allows users to create and modify network topologies by adding routers, links, and network parameters. Through this dashboard, users can configure different scenarios such as link failures, router failures, congestion levels, packet loss percentages, and link costs.
- Simulation Control Layer: Responsible for managing the overall operation of the simulator. It keeps track of the current state of the network and controls when different events occur. For example, if a user wants to simulate a link failure after a certain time interval or increase traffic on a particular link, this layer schedules and executes those actions. It also stores predefined network scenarios so that experiments can be repeated and compared easily. This layer ensures that all modules work together in a coordinated manner during simulation.
- RIP Engine: Simulates the behavior of the Routing Information Protocol (RIP), which is a Distance Vector routing protocol. Each router maintains a routing table containing information about reachable destinations and their corresponding costs. Routers periodically exchange their routing tables with neighboring routers. Using the Bellman-Ford algorithm, each router updates its routing information whenever a better path is discovered. The process continues until all routers agree on the shortest available paths and the network reaches convergence.
- OSPF Engine: Simulates the Open Shortest Path First (OSPF) routing protocol, which is based on Link State Routing. Instead of exchanging complete routing tables, routers share information about their directly connected links using Link State Advertisements (LSAs). Using this information, each router constructs a complete view of the network topology and independently computes the shortest paths to all destinations using Dijkstra's algorithm.
- Failure / Loss / Congestion Module: This module is responsible for introducing real-world network conditions into the simulation. It can simulate link failures, router failures, packet loss, and network congestion. When a link or router fails, the routing protocols must recalculate routes and find alternative paths. The module can also assign different traffic loads to network links to create congestion scenarios. These simulations help evaluate the robustness and adaptability of routing protocols under challenging network conditions.
- Performance Analysis Engine: Evaluates how well the routing protocols perform under different scenarios. It collects data from the RIP and OSPF modules and calculates various performance metrics such as path cost, hop count, recovery time after failures, throughput, packet loss, and delay. The results are used to compare the efficiency of both routing protocols.
- Protocol Recommendation Engine: Based on factors such as network size, traffic load, frequency of failures, and congestion levels, it recommends the most suitable routing protocol for a given network scenario. The recommendation is generated using a predefined set of rules and performance observations collected during simulation. Along with the recommendation, the module also provides a justification explaining why a particular protocol is more suitable.

- Visualization & Validation Layer: This layer is responsible for presenting simulation results in a clear and interactive manner. Using NetworkX and Matplotlib, it displays network topologies, routing paths, packet flow movements, and performance graphs..

6. Methodology / Module Description

The system is decomposed into the following functional modules, each with a clearly bounded responsibility so that RIP, OSPF, failure handling, and analysis can be developed and tested independently before integration.

Module	Core Function
Topology Builder	Constructs router/link graph models used by all downstream modules.
RIP Engine	Distance Vector routing via Bellman-Ford; periodic table exchange and updates.
OSPF Engine	Link State routing via Dijkstra; LSA flooding and SPF tree computation.
Visualization Layer	Graph-based rendering of topology, routing paths, and packet animation.
Failure Simulator	Injects link/router failures; triggers re-convergence and recovery tracking.
Congestion Engine	Assigns and updates traffic loads; biases route selection away from congestion.
Loss & Cost Modeler	Applies packet loss percentages and dynamic bandwidth/delay/cost changes.
Performance Analyzer	Computes path cost, hop count, convergence time, recovery time, throughput, delay.
Recommendation Engine	Rule-based comparison of RIP vs OSPF against live network conditions.
Validation Layer	Cross-checks simulator output against Wireshark captures and Packet Tracer runs.

7. Tools Used for Implementation

Tool / Library	Purpose
Python 3.x	Core implementation language for simulation and analysis logic.
NetworkX	Graph representation of network topology; shortest-path utilities.
Matplotlib	Rendering topology diagrams, packet-flow visuals, and performance graphs.
NumPy	Numerical computation for performance metrics and randomized traffic/loss models.
Pandas	Tabular storage and analysis of routing tables and comparison datasets.
Wireshark	Capture and inspection of real routing/ICMP traffic for validation.
Cisco Packet Tracer	Equivalent RIP/OSPF topology construction for cross-verification.
Visual Studio Code	Development environment for implementation and debugging.